

Competitive Programming

Section 56 Editorial: Hard Greedy And Matroids Intro

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. Deadline Profit	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
3	B. Minimum Interval Cover	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
4	C. MST Savings	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	8
5	What to carry forward	10

1 Editorial overview

1.1 Detailed solution guide

A. Scheduling with Deadlines. Sort jobs by deadline and keep the selected profits in a min-heap. After considering all jobs with deadline at most d , the heap contains the best set of jobs that can fit into d slots. If adding a job creates too many selected jobs, remove the smallest profit. This exchange is safe because any feasible schedule with the low-profit job can replace it by the higher-profit job without using more time.

B. Interval Cover. To cover a target segment, repeatedly among all intervals starting at or before the current uncovered point choose the one with farthest right endpoint. Any valid solution must choose some interval covering the current point; choosing the farthest-reaching such interval cannot make the remaining suffix harder to cover.

C. Kruskal Savings. Sort edges by weight and add an edge only if it connects two different components. The DSU maintains the connected components of the partial forest. The cut property proves each chosen edge is safe: it is the lightest available edge crossing some cut. Savings are total edge weight minus the MST weight.

Common pitfalls. Greedy proofs need an exchange or cut argument, not just intuition. For deadline scheduling, deadlines are capacities. For interval cover, detect gaps when no candidate reaches the current point.

The three greedy solutions use exchange arguments.

- A keeps the highest-profit feasible set of unit jobs by deadline using a min-heap.
- B repeatedly chooses, among intervals that already reach the uncovered point, the one extending farthest right.
- C is Kruskal's algorithm: add an edge exactly when it connects two DSU components, then compute removable weight.

Each proof compares the greedy choice with an optimal solution and shows the greedy choice can be exchanged in without worsening feasibility or objective.

2 A. Deadline Profit

2.1 Restatement

Each unit job has deadline and profit. Maximize profit.

2.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

2.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

2.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

2.6 Full Python solution

```
import sys,heapq
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n=data[0]; jobs=sorted((data[i],data[i+1]) for i in range(1,2*n+1,2)); h=[]
    for d,p in jobs:
```

```
        heapq.heappush(h,p)
        if len(h)>d: heapq.heappop(h)
    print(sum(h))
if __name__=="__main__": main()
```



```
    best=cur
    while i<n and seg[i][0]<=cur: best=max(best,seg[i][1]); i+=1
    if best==cur: print(-1); return
    cur=best; ans+=1
print(ans)
if __name__=="__main__": main()
```

4 C. MST Savings

4.1 Restatement

Print total edge weight removable while keeping the graph connected, or -1 .

4.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

4.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

4.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;if(!(ci
```

4.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m=data[0],data[1]; edges=[]; total=0; idx=2
    for _ in range(m):
```

```

        a,b,w=data[idx],data[idx+1],data[idx+2]; idx+=3; edges.append((w,a-1,b-1)); total+=w
p=list(range(n))
def f(x):
    while p[x]!=x: p[x]=p[p[x]]; x=p[x]
    return x
keep=0; cnt=0
for w,a,b in sorted(edges):
    ra,rb=f(a),f(b)
    if ra!=rb: p[ra]=rb; keep+=w; cnt+=1
print(total-keep if cnt==n-1 else -1)
if __name__=="__main__": main()

```

5 What to carry forward

Red-level improvement comes from making the invariant explicit, stress-testing it, and then upsolving the stronger external versions.